

NEXUSAGENT

Rapport d'Analyse Complet

Analyse fichier par fichier (hors tests)

Projet : github.com/DEVGD-glitch/NexusAgent

Branche : master

Version : 0.1.0

Date : 13 mai 2026

Genere par Z.ai

Table des Matieres

1. Vue d'ensemble du Projet

- 1.1 Identite et objectifs
- 1.2 Architecture globale
- 1.3 Stack technologique

2. Analyse des Fichiers Racine

- 2.1 Configuration et dependances
- 2.2 Scripts de lancement
- 2.3 Documentation

3. Module Core (nexus/core/)

- 3.1 Configuration et DI
- 3.2 Exceptions et messages d'erreur
- 3.3 Registre et superviseur
- 3.4 A2A, observabilite et telemetrie

4. Module LLM (nexus/llm/)

- 4.1 Routeur LLM
- 4.2 Chaîne de fallback
- 4.3 Fournisseurs LLM

5. Module Agents (nexus/agents/)

- 5.1 Agent de base (BaseAgent)
- 5.2 Agents specialises
- 5.3 Couche OpenAI Agents

6. Module Memoire (nexus/memory/)

- 6.1 Systeme memoire 5 niveaux
- 6.2 Service ChromaDB
- 6.3 Orchestrateur et compacteur

7. Module Securite (nexus/security/)

- 7.1 Guardrails et permissions
- 7.2 Sandbox et vault
- 7.3 Audit et rate limiting

8. Modules Knowledge et Orchestrator

- 8.1 Recherche profonde et graphe de connaissance
- 8.2 Moteurs d'orchestration
- 8.3 Motifs et cycle de vie des competences

9. Modules Reasoning, Dev, API, Comms

9.1 Raisonnement (ReAct, ToT, LATS)

9.2 Developpement et execution de code

9.3 Passerelle API

9.4 Communication et avatar

10. Modules Computer, Browser, MCP, CLI

10.1 Automatisation ordinateur

10.2 Navigateur et MCP

10.3 CLI et outils MCP

11. Frontend (nexus-web/)

12. Synthese des Bugs Critiques

13. Recommandations Prioritaires

1. Vue d'ensemble du Projet

1.1 Identite et objectifs

NexusAgent (NEXUS) se presente comme un agent IA souverain, avec pour slogan Zero Cloud, Zero Compromis. Le projet est entierement redige en francais, ce qui constitue une singularite notable pour un projet open-source de cette envergure. L'objectif affirme est de depasser des concurrents comme Hermes Agent, OpenClaw, Space Agent et Bolt.diy, avec un plan ambitieux de 11 phases de developpement dont seule la phase 1 est completee.

Le projet revendique 30 000+ lignes de code Python, 22 000+ lignes de tests, 110+ fichiers Python repartis en 22 modules, et 8 fournisseurs LLM dont 3 gratuits sans cle API. La version actuelle est 0.1.0, ce qui indique un stade tres precocce malgre l'ambition affichee. Le depot GitHub contient des placeholders YOUR_USERNAME dans les URLs, suggerant que le projet n'est pas encore pret pour une publication publique.

1.2 Architecture globale

L'architecture de NEXUS s'articule autour de 22 modules organises en couches distinctes. Le module core fournit la configuration, l'injection de dependances, le registre d'agents et la supervision. Le module llm gere le routage multi-fournisseur avec fallback automatique. Le module memory implemente un systeme memoire hierarchique a 5 niveaux (travail, episodique, semantique, procedural, identite) soutenu par ChromaDB. Le module orchestrator propose 3 moteurs (LangGraph, CrewAI, Google ADK) avec 6 motifs de coordination. Le module security offre 6 couches complementaires (sandbox, audit, guardrails, permissions, rate limiting, vault).

Module	Fichiers	Lignes	Role principal
core/	15	~2500	Config, DI, registre, exceptions, A2A
llm/	14	~2800	Routeur 13 fournisseurs, fallback, circuit breaker
agents/	7	~3600	5 agents specialises + couche OpenAI
memory/	9	~2200	Memoire 5 niveaux + ChromaDB + orchestrateur
security/	8	~2400	Sandbox, guardrails, vault, audit, permissions
knowledge/	6	~1800	Recherche, graphe, RAG, watchdog, web search
orchestrator/	7	~2200	3 moteurs, 6 motifs, cycle de vie competences
reasoning/	5	~1200	ReAct, Tree-of-Thought, LATS
dev/	6	~1600	Moteur de code, execution, deploiement, git
api/	3	~1100	Passerelle FastAPI, proxy Puter

comms/	10	~2500	Telegram, email, voix, avatar VRM
computer/	5	~1500	PyAutoGUI, OCR, controle GUI, processus
browser/	3	~800	Service Docker, Playwright
mcp_server + mcp_tools/	14	~1800	43+ outils MCP, serveur FastMCP
cli/	2	~600	Interface Typer + Rich

Tableau 1 : Vue d'ensemble des modules du projet NEXUS

1.3 Stack technologique

Le backend utilise Python 3.11+ avec FastAPI, Pydantic 2.11+, ChromaDB 1.0+, LangGraph 0.4+, LiteLLM 1.67+ et Typer pour la CLI. Le frontend est construit avec Next.js 16, React 19, shadcn/ui, Zustand et Framer Motion. L'application desktop utilise Tauri v2. L'orchestration Docker comprend 3 services (nexus-core, chromadb, browser-service). Les dependances optionnelles incluent CrewAI, Google ADK, OpenAI Agents SDK, Playwright et AIAvatarKit.

[CRITIQUE] Inconsistance majeure : pyproject.toml et requirements.txt definissent des ensembles de dependances et des contraintes de version differents. L'installation via pip install -e . vs pip install -r requirements.txt produit des environnements differents.

2. Analyse des Fichiers Racine

2.1 Configuration et dependances

Le fichier pyproject.toml definit le paquet nexus-agent v0.1.0 avec 23 dependances principales et des groupes optionnels (browser, desktop, avatar, multiagent, dev). On note des redundances : customtkinter, Pillow et httpx apparaissent a la fois dans les dependances principales et les groupes optionnels. Le fichier requirements.txt contient 50+ paquets avec des contraintes de version differentes de pyproject.toml (par exemple langgraph est 0.4+ dans pyproject.toml mais inferieur a 1.0 dans requirements.txt). Les paquets de test (pytest, pytest-asyncio, pytest-cov) sont dans requirements.txt mais auraient du rester dans le groupe dev uniquement.

Le fichier .env.example definit la configuration environnementale avec 13 fournisseurs LLM organises en 3 tiers (gratuits, payants, locaux). Le routeur par default est Gemini avec une chaine de fallback vers 7 fournisseurs. Cependant, le port par default est 8080 dans .env.example alors que les scripts de lancement utilisent 8081 (car le port 8080 est pris par Windows svchost). La cle secrete par default 'change-me-to-a-secure-random-string' represente un risque si les utilisateurs ne la modifient pas.

2.2 Scripts de lancement

start.sh est le script Unix principal : il cree un venv si absent, installe les dependances, verifie Node.js, copie .env.example vers .env si necessaire, lance le backend sur le port 8081 et le frontend sur le port 3000. start_web.sh est une version simplifiee sans verification de dependances. Les deux scripts utilisent des handlers trap pour l'arret propre. Le script run_nexus.py est un point d'entree alternatif qui ajoute la racine du workspace au sys.path et charge le fichier .env via dotenv.

[AVERTISSEMENT] Port inconsistent : .env.example definit PORT=8080, mais start.sh et start_web.sh codent en dur le port 8081. Cette incoherence peut causer des erreurs de connexion.

2.3 Documentation

Le README.md presente le projet de maniere attractive mais contient des placeholders YOUR_USERNAME dans les URLs GitHub, le rendant inutilisable pour le clonage public. Le CHANGELOG.md est severement obsolete : la version 1.0.0 datee de 2025-01-01 decrit un projet beaucoup plus simple (OpenAI + Anthropic uniquement), sans reflechir les ajouts massifs recents. Le AGENTS.md est le document technique le plus detaille avec 12 points d'entree, l'architecture complete et les conventions de codage, mais il contient un arbre d'architecture duplique par erreur de copier-coller. PLAN.md decrit 11 phases ambitieuses, dont seule la phase 1 est completee.

3. Module Core (nexus/core/)

3.1 Configuration et DI

Le fichier config.py implemente le pattern Settings Object avec pydantic-settings. La classe NexusConfig contient 50+ champs de configuration organises en 13 categories, charges depuis les variables d'environnement et les fichiers .env. La fonction get_settings() utilise lru_cache pour fournir un singleton efficace avec possibilite de rechargement via reload_settings(). La validation de production bloque l'utilisation de la cle secrete par default en mode production.

[CRITIQUE] Bug : available_providers inclut toujours ollama, pollinations, g4f et deepinfra, meme s'ils ne sont pas joignables. Cela donne une fausse impression de disponibilite et peut fausser le routage LLM.

Le fichier di.py implemente un conteneur d'injection de dependances minimaliste avec 95 lignes. Le DIContainer utilise des fabriques et des singletons, mais toutes les fabriques deviennent des singletons apres la premiere resolution, sans possibilite d'instances transitoires. Le conteneur n'est pas thread-safe et configure_container() echoue bruyamment si un module est manquant.

3.2 Exceptions et messages d'erreur

Le fichier `exceptions.py` definit une hierarchie d'exceptions structuree avec 6 categories principales (Configuration, Memory, LLM, Orchestrator, Security, Browser) et des sous-types specifiques. Chaque exception porte un code, des details contextuels et une methode `to_dict()` pour la serialisation API. C'est un excellent design qui facilite le debug et la reponse API. Cependant, des categories manquent : `KnowledgeError`, `CommsError`, `DesktopError` et `CLLError` n'existent pas.

Le fichier `error_messages.py` transforme les exceptions techniques en messages conviviaux en francais via `ERROR_MAP`. Le systeme utilise un fallback a 3 niveaux : correspondance directe, correspondance par motif, message generique. Probleme : `ERROR_MAP` mute les entrees en place (effet de bord sur un global partage), et plusieurs types d'exceptions ne sont pas mappees.

3.3 Registre et superviseur

[CRITIQUE] Bug critique dans `registry.py` : `spawn()` cree un `AgentInstance` mais n'appelle jamais la fabrique pour creer l'agent reel. Les 5 types par default sont enregistres avec `factory=None`. Le registre est un systeme de suivi sans creation effective d'agents.

Le superviseur (`supervisor.py`) est un stub : `_monitor_loop()` est un no-op avec le commentaire Future: Implement actual health checks. Le superviseur enregistre les services mais ne verifie jamais leur sante et ne les redemarre pas.

3.4 A2A, observabilite et telemetrie

Le module A2A (`a2a.py`) implemente le protocole Agent-to-Agent avec decouverte via `/.well-known/agent.json`, delegation de taches avec polling et messagerie inter-agents. Le module d'observabilite (`observability.py`) fournit du tracing en memoire avec Span et Tracer, mais sans export vers OpenTelemetry ou Langfuse malgre les references dans la configuration. Le module de telemetrie (`telemetry.py`) offre du crash reporting opt-in avec sanitization PII, mais les rapports ne sont jamais envoyes et restent locaux.

[AVERTISSEMENT] Imports circulaires dans `a2a.py` : `TaskDelegate.wait_for_completion()` importe `A2AProtocol` depuis le meme module.

4. Module LLM (nexus/llm/)

4.1 Routeur LLM

Le routeur LLM (`router.py`, 818 lignes) est le coeur du sous-systeme LLM. Il gere 13 fournisseurs, 2 modes de routage (fournisseur unique avec retry, ou auto-routage par complexite), et une double strategie d'appel (LiteLLM puis fallback HTTP direct). Gemini est specialise avec un appel direct bypassant LiteLLM en raison d'erreurs 500 intermittentes. Le routeur implemente egalement une boucle d'appel d'outils avec `max_tool_turns=10`.

[CRITIQUE] Bug : `_call_glm_direct` et `_call_ollama_direct` sont du code mort jamais appele depuis `_call_provider`. Les methodes existent mais ne sont jamais utilisees.

[AVERTISSEMENT] Le streaming est un no-op : `_call_provider_streaming` delegue au non-streaming et retourne le contenu. Commentaire : For MVP.

4.2 Chaine de fallback

La chaine de fallback (`fallback.py`, 520 lignes) ajoute un circuit breaker, un suivi de sante des fournisseurs et des politiques de retry par-dessus le `LLMRouter`. Apres 3 echecs consecutifs, un fournisseur est marque `UNHEALTHY` pendant 60 secondes. La latence moyenne est calculee sur les 100 derniers echantillons.

[AVERTISSEMENT] `FallbackChain` cree un nouveau `LLMRouter` par tentative, ce qui est couteux et empeche le pooling de connexions. La logique `_is_provider_available` est dupliquee entre `FallbackChain` et `LLMRouter` et peut diverger.

4.3 Fournisseurs LLM

Les 13 fournisseurs sont implements avec un pattern coherent : catalogue de modeles, dataclass de reponse, methodes `complete/stream/stats`. Les fournisseurs payants (OpenAI, Anthropic, Gemini, GLM) utilisent `LiteLLM` puis fallback HTTP direct. Les fournisseurs gratuits (Pollinations, G4F, DeepInfra) utilisent le routeur `FreeProviderRouter` avec fallback Pollinations vers G4F.

[CRITIQUE] Bug dans `free_router.py` : `G4F_MODELS` est reference mais jamais importe, causant un `NameError` a l'execution.

[CRITIQUE] Bug : Les fournisseurs gratuits appellent `LLMProviderError` avec un seul argument string au lieu des keyword args attendus (`provider`, `reason`, `model`). Cela peut echouer a l'execution.

Le fournisseur Ollama (`ollama_provider.py`) a une methode `is_available()` qui retourne toujours `True`, meme si le serveur Ollama n'est pas en cours d'execution. Le routeur tentera Ollama meme quand il est indisponible, menant a des erreurs de connexion.

5. Module Agents (nexus/agents/)

5.1 Agent de base (BaseAgent)

Le `BaseAgent` (`base.py`, 966 lignes) est la classe abstraite pour tous les agents NEXUS. Il implemente le cycle Plan-Execute-Reflect avec des services a initialisation paresseuse (`llm_router`, `memory`, `security`, `mcp_client`). L'agent gere 32 outils MCP via un dictionnaire hardcoded et 6 outils fallback. La methode `_call_llm()` implemente une boucle d'appel d'outils complete.

[CRITIQUE] `execute_with_fallback` ne crée pas réellement d'agent de fallback : même si `fallback_agent_type` est fourni, la méthode retourne juste FAILED avec un message de suggestion.

[AVERTISSEMENT] La carte des outils MCP est un dictionnaire hardcodé de 32 entrées, fragile et verbeux. Environ 150 lignes de wrappers quasi-identiques. Aucune gestion de la fenêtre de contexte : la liste `AgentContext.conversation` croît sans limite.

5.2 Agents spécialisés

Le `ResearcherAgent` (`researcher.py`) décompose les questions de recherche, effectue des recherches web et approfondies, puis synthétise avec citations. Le `DeveloperAgent` (`developer.py`) propose 3 modèles de plan (standard, debugging, review) sélectionnés par mots-clés. L'`AnalystAgent` (`analyst.py`) génère et exécute du code Python d'analyse. L'`OperatorAgent` (`operator.py`, 671 lignes) est le plus complexe avec 5 modèles de plan et un mode ADVISORY qui génère des plans/scripts sans exécution automatique d'opérations destructrices.

[AVERTISSEMENT] Le `DeveloperAgent` utilise un regex simpliste pour détecter les chemins de fichiers : `re.search(r'[\w/\-\.]+\.\w+', task)` qui matche aussi `python3.11` ou e.g.

5.3 Couche OpenAI Agents

La couche OpenAI Agents (`openai_layer.py`, 793 lignes) intègre l'OpenAI Agents SDK avec handoffs, guardrails et tracing. Si le SDK n'est pas disponible, le système bascule vers une exécution native NEXUS avec des handoffs simulés. Le `HandoffRouter` utilise un routage par mots-clés, et les guardrails s'intègrent avec la couche de sécurité NEXUS.

[CRITIQUE] Bug : `_run_native` a la configuration de handoff en pass (ligne 638-642). Les handoffs ne fonctionnent jamais en mode natif.

[AVERTISSEMENT] Pas de détection de cycles dans la boucle de handoff : si A délègue à B et B délègue à A, la boucle rebondit `max_handoffs` fois sans progresser.

6. Module Mémoire (nexus/memory/)

6.1 Système mémoire 5 niveaux

Le système mémoire implémente 5 niveaux hiérarchiques. L1 Working Memory (`working.py`) gère la fenêtre de contexte active avec un budget de 30K tokens, une éviction par priorité et une compression automatique. L2 Episodic Memory (`episodic.py`) stocke le journal chronologique des expériences. L3 Semantic Memory (`semantic.py`) gère les faits de connaissance et les chunks RAG. L4 Procedural Memory (`procedural.py`) cristallise les compétences extraites des trajectoires réussies. L5 Identity Memory (`identity.py`) stocke les profils utilisateur et les préférences.

Niveau	Classe	Namespace	Role
L1	WorkingMemory	(en memoire)	Contexte actif, budget tokens
L2	EpisodicMemory	episodes	Journal des experiences
L3	SemanticMemory	knowledge	Faits et chunks RAG
L4	ProceduralMemory	skills	Competences cristallisees
L5	IdentityMemory	identity	Profils et preferences

Tableau 2 : Systeme memoire a 5 niveaux

[CRITIQUE] Bug dans working.py : `__post_init__` ne declenche jamais le fallback vers les settings car `max_tokens=30000` par default et la condition est `if self.max_tokens <= 0`. La condition devrait etre `if self.max_tokens is None`.

[CRITIQUE] Bug dans identity.py : Impossible de definir `language_preference='en'` sur merge car 'en' est la valeur par default et sert de sentinelle. Meme probleme pour `communication_style='professional'`.

[AVERTISSEMENT] Bug dans episodic.py : `recall_recent` utilise `results['documents'][i]` (index plat) tandis que `recall_similar` utilise `results['documents'][0][i]` (index imbrique), refletant des formats de reponse ChromaDB differents.

6.2 Service ChromaDB

Le `NexusMemoryService` (`chroma_service.py`) fournit le backend vectoriel avec 6 namespaces valides, un cache de client, une injection automatique de metadonnees et une deduplication par hash SHA-256. Le service gere les upsert sur ID existant et utilise un verrou async pour la concurrence.

[CRITIQUE] Bug : `_cache_lock` est cree mais jamais utilise dans `_get_cached_client`, permettant la creation de clients dupliques.

[AVERTISSEMENT] `update()` avec texte remplace toutes les metadonnees au lieu de les fusionner, ce qui peut perdre des champs comme `source` et `type`.

6.3 Orchestrateur et compacteur

[CRITIQUE] Bug critique dans `orchestrator.py` : `recall()` pour WORKING memory cree un nouveau `WorkingMemory` vide au lieu d'utiliser `self._working_sessions[session_id]`. Le rappel ne retourne jamais rien.

[CRITIQUE] Bug critique dans `orchestrator.py` : Le store semantique utilise le namespace 'semantic' mais `SemanticMemory` utilise 'knowledge'. Les donnees sont orphelines.

Le `MemoryCompactor` (`compactor.py`) offre la compression, la deduplication et la detection de contradictions. Cependant, `detect_contradictions` est $O(n^2)$ avec une limite arbitraire de 200, et la

detection est naive (mots de negation + chevauchement de 5+ mots).

7. Module Securite (nexus/security/)

7.1 Guardrails et permissions

Le module de guardrails (guardrails.py) implemente la validation d'entree/sortie avec 4 guardrails : PromptInjectionGuardrail (14 motifs regex), PIIGuardrail (email, telephone, CB, SSN, IP), ContentModerationGuardrail (violence, auto-mutilation, discours de haine), et OutputValidationGuardrail (fuite de cles API). Le GuardrailManager orchestre le pipeline avec redaction cumulative PII.

[AVERTISSEMENT] Les motifs d'injection par prompt sont facilement contournables par obfuscation, astuces Unicode ou nouvelles formulations. Les patterns PII sont centes US (pas de numeros internationaux). La redaction d'adresses IP est trop agressive (matche les numeros de version).

Le module de permissions (permissions.py) offre 2 modes (AUTO et CONFIRM) avec 9 types d'actions. Le mode AUTO n'approuve que les actions non dangereuses, le mode CONFIRM demande confirmation pour tout. Cependant, PermissionResult est defini mais jamais utilise : le systeme peut demander une permission mais n'a pas de moyen de recevoir la reponse programmatiquement.

7.2 Sandbox et vault

Le LocalSandbox (sandbox.py) offre l'execution de code isolee avec un blocklist de motifs dangereux, des restrictions d'import et des limites de ressources. Le DockerSandbox ajoute l'isolation complete (pas de reseau, FS en lecture seule, capacites supprimees, utilisateur non-root). Le PerActionSandbox (docker/per_action_sandbox.py) cree des conteneurs ephemeriques par action.

[AVERTISSEMENT] Le sandbox local base sur un blocklist est intrinsequement contournable (ex: `getattr(__builtins__, '__imp' + 'ort__')('os'))`). La documentation recommande correctement le DockerSandbox pour la production.

[AVERTISSEMENT] Le fichier pepper du vault n'a pas de permissions restreintes sur aucune plateforme. Le vault stocke des donnees dans `~/.nexus/vault` qui peut etre sur NFS ou un filesystem partage.

7.3 Audit et rate limiting

L'AuditLogger (audit.py) fournit un journal d'audit immutable avec rotation quotidienne et requetes. Le RateLimiter (rate_limiter.py) implemente un algorithme de fenetre glissante avec burst allowance et 4 limites preconfigurees.

[AVERTISSEMENT] Le RateLimiter utilise `threading.Lock` qui peut bloquer l'event loop dans un contexte async. La memoire des fenetres croit sans limite.

8. Modules Knowledge et Orchestrator

8.1 Recherche profonde et graphe de connaissance

Le DeepResearch (deep_research.py) decompose un sujet en sous-requetes, recherche iterativement dans ChromaDB et le web, puis synthetise un rapport structure. Cependant, _extract_finding ne fait que prefixer le snippet avec le titre au lieu d'utiliser le LLM comme indique dans la docstring. Le KnowledgeGraph (knowledge_graph.py) utilise NetworkX avec double index (_entity_index et _rel_type_index) pour un acces O(1) aux entites.

[CRITIQUE] Bug : _rel_type_index n'est pas mis a jour lors de remove_entity() ni reconstruit lors de load(), creant des references pendantes.

[AVERTISSEMENT] Le RAG pipeline (rag_pipeline.py) n'utilise jamais le graphe de connaissance malgre la promesse Multi-source retrieval (ChromaDB + Knowledge Graph). Le reranker est juste un sorted() par score, pas un vrai cross-encoder.

Le WatchdogService (watchdog.py) offre la surveillance RSS, la detection de changements web et les recherches periodiques avec planification cron-like. Le parsing RSS/Atom base sur des regex est fragile et le cron parser est naif. Le MultiSourceWebSearch (web_search.py) agrege les resultats de 4 moteurs (z-ai-sdk, SerpAPI, Brave, DuckDuckGo) avec fallback en chaine.

[AVERTISSEMENT] Le scraping DuckDuckGo via regex est extremement brittle et cassera sans avertissement quand DDG change ses classes CSS.

8.2 Moteurs d'orchestration

Le LangGraph Engine (langgraph_engine.py) est le moteur principal avec un cycle Plan-Execute-Reflect via StateGraph. Cependant, run_nexus_task() retourne via _run_simple_loop, rendant tout le code LangGraph (lignes 557-596) inaccessible. Le commentaire indique Use simple loop for now (LangGraph graph has state update issues). L'integration LangGraph est effectivement desactivee.

[CRITIQUE] Bug critique : Le code LangGraph dans run_nexus_task() est du code mort (inaccessible apres return). L'integration LangGraph est desactivee et non testee.

[CRITIQUE] Bug : fn_name potentiellement non defini dans executor_node si tool_calls est vide.

[CRITIQUE] Bug : _execute_tool_call code en dur http://127.0.0.1:8081 comme URL du backend, sans configuration possible.

Le CrewAI Engine (crewai_engine.py) integre le framework CrewAI avec 5 templates d'agents (researcher, developer, analyst, writer, manager) et un fallback natif NEXUS. L'ADK Engine (adk_engine.py) integre le Google Agent Development Kit avec gestion de sessions en memoire. Les deux moteurs suivent le pattern try-native-then-fallback.

[AVERTISSEMENT] Le fallback CrewAI est purement sequentiel meme si `async_execution=True`.
L'ADK Engine tronque le contexte de session a 2000 caracteres.

8.3 Motifs et cycle de vie des competences

Le module `patterns.py` implemente 6 motifs de coordination multi-agent (Supervisor, Pipeline, Parallel, Hierarchical, Mesh, Swarm). Cependant, `supervisor_pattern` est fonctionnellement identique a `parallel_pattern` (les deux executent en parallele avec `asyncio.gather`), et `mesh_pattern` execute les agents sequentiellement au lieu de les paralleliser dans chaque round.

[CRITIQUE] Bug dans `skill_lifecycle.py` : La validation n'execute pas reellement le code genere. `_run_test_case` envoie l'input au LLM au lieu de lancer le code. Le code completement casse passe la validation.

[AVERTISSEMENT] `deploy_skill` stocke la competence dans ChromaDB mais ne l'enregistre jamais comme outil MCP.

9. Modules Reasoning, Dev, API, Comms

9.1 Raisonnement (ReAct, ToT, LATS)

Le module de raisonnement propose 3 strategies. ReAct (`react.py`) implemente la boucle Thought-Action-Observation mais n'execute pas reellement les outils malgre le parametre `tools` accepte. Le selector (`selector.py`) choisit la strategie par mots-cles avec fallback vers ReAct.

[CRITIQUE] Bug dans ReAct : Le parametre `tools` est accepte mais completement ignore. La boucle n'a aucune capacite d'execution d'outils.

Tree-of-Thought (`tot.py`) implemente le beam search sur des pensees candidates avec evaluation LLM. Mais le parsing du score prend le premier chiffre (8.5 devient 8, 10 devient 1). LATS (`lats.py`) implemente Monte Carlo Tree Search avec UCB1, mais une simulation coute 6-8 appels LLM, et 20 simulations representent 120-160 appels LLM, ce qui est tres couteux.

[CRITIQUE] Bug dans LATS : `rollout_node` peut etre non defini dans la branche else de l'etape de simulation, causant un `UnboundLocalError`.

9.2 Developpement et execution de code

Le CodeEngine (`code_engine.py`) offre la generation, revue et refactoring de code avec support CodeAct (generation-execution-feedback iteratif). La revue combine des patterns statiques (regex pour `eval()`, `exec()`, `pickle.load()`, injection SQL) et une revue semantique LLM. Le CodeExecutor (`code_executor.py`) supporte 3 backends (local, Docker, remote) avec stripping des cles API dans l'environnement d'execution.

[AVERTISSEMENT] CodeAct : La detection de completion est naive : if 'TASK_COMPLETE' in result.stdout or 'done' in result.stdout.lower(). Le mot 'done' dans toute sortie de programme termine prematurement la boucle.

9.3 Passerelle API

La passerelle API (api/gateway.py, ~1000+ lignes) expose toutes les capacites NEXUS via FastAPI avec CORS, middleware d'authentification et rate limiting. Elle definit 27+ handlers d'outils couvrant la memoire, le graphe de connaissance, le systeme de fichiers, l'execution de code, la recherche web et l'avatar.

[CRITIQUE] Bug : `_tool_reason_react` importe `ReactReasoner` qui n'existe pas dans `react.py` (c'est `ReActLoop`). `ImportError` a l'execution.

[CRITIQUE] Securite : La comparaison de token utilise `!=` au lieu de `hmac.compare_digest()`, vulnérable aux attaques temporelles.

[AVERTISSEMENT] `_tool_install_package` permet l'installation arbitraire de paquets pip sans allowlist ni verification. Risque de securite majeur.

9.4 Communication et avatar

Le module comms implemente Telegram (`telegram_bot.py`), email/calendar via Gmail API (`email_calendar.py`), et voix via Whisper/TTS OpenAI (`voice_io.py`). Le sous-module avatar offre un pipeline Speech-to-Speech complet : VAD, STT, LLM, TTS via VOICEVOX, lip sync et rendu VRM Three.js.

[CRITIQUE] Bug dans `vrn_renderer.py` : Le serveur WebSocket n'est jamais demarre. Le rendu VRM est non fonctionnel.

[AVERTISSEMENT] `VoiceIO` : `stream_to_file()` est synchrone dans une methode async, bloquant l'event loop. Pas de refresh OAuth2 pour les tokens email/calendar.

10. Modules Computer, Browser, MCP, CLI

10.1 Automatisation ordinateur

Le module computer fournit l'automatisation desktop via PyAutoGUI + Tesseract OCR (`computer_use.py`), le controle GUI multi-plateforme avec adaptateurs AT-SPI/UIAutomation (`gui_control.py`), la gestion de processus (`process_manager.py`) et la comprehension d'ecran par OCR + Vision (`screen_understanding.py`).

[CRITIQUE] Bug dans `screen_understanding.py` : `b64[:10000]` tronque les donnees base64 dans les appels vision, corrompant les images. Le base64 ne doit jamais etre tronque arbitrairement.

[AVERTISSEMENT] Duplication massive entre ComputerUse et GUIController : click, double_click, type_text, hotkey, scroll, capture_screen, etc. sont quasi-identiques. tempfile.mktemp() est un risque de securite (race condition).

10.2 Navigateur et MCP

Le BrowserService (browser_service.py) est un client HTTP pour un micro-service Docker browser-use, avec fallback vers le scraping basique. PlaywrightExtensions (playwright_ext.py) offre l'automatisation native Playwright avec selecteurs auto-cicatrisants (4 niveaux : CSS, texte, role, texte partiel).

Le serveur MCP (mcp_server.py) expose 43+ outils via FastMCP avec 3 ressources et 3 templates de prompt. Cependant, les outils avatar sont importes mais jamais enregistres dans le serveur MCP, et plusieurs outils sont des stubs retournant not_implemented (reason_tot, reason_lats, run_supervisor, run_swarm, web_screenshot).

10.3 CLI et outils MCP

La CLI (cli/app.py) utilise Typer + Rich avec les commandes run, chat, status, config, serve, agents, skills, eval, context7 et memory. La commande chat appelle asyncio.run() dans un contexte potentiellement deja en boucle d'evenements, ce qui peut crasher en Python 3.10+.

[AVERTISSEMENT] Les outils MCP ont plusieurs problemes : llm_stream ne streame pas reellement, search_files utilise grep -r avec risque d'injection de commande, et install_package permet l'installation arbitraire de paquets.

11. Frontend (nexus-web/)

Le frontend est une application Next.js 16 avec React 19, shadcn/ui, Zustand pour l'etat global et Framer Motion pour les animations. L'interface propose 8 panneaux (Agents Wall, Chat, Tasks, Code, Memory, Knowledge, Tools, Settings) accessibles via une barre laterale animee. Le chat-panel supporte le Markdown, la coloration syntaxique, un avatar VRM avec expressions et animation de pensee, et un flux d'activite en temps reel.

L'API proxy (route.ts) redirige les requetes vers le backend Python a http://127.0.0.1:8081. Le client API (nexus-api.ts) offre des methodes typees pour tous les endpoints backend. Le store Zustand (nexus-store.ts) gere l'etat global avec un journal d'activite limite a 50 entrees.

[AVERTISSEMENT] L'URL du backend est code en dur dans route.ts (http://127.0.0.1:8081). Les headers d'authentification ne sont pas transferees. Le texte de l'interface est en francais code en dur sans framework i18n.

12. Synthese des Bugs Critiques

L'analyse a revele un nombre significatif de bugs critiques qui affectent le fonctionnement du projet. Le tableau suivant resume les bugs les plus importants, classes par severite.

Severite	Module	Bug
CRITIQUE	orchestrator.py	recall() WORKING cree un WorkingMemory vide
CRITIQUE	orchestrator.py	Store semantique namespace 'semantic' vs 'knowledge'
CRITIQUE	langgraph_engine.py	Code LangGraph inaccessible (apres return)
CRITIQUE	langgraph_engine.py	fn_name potentiellement non defini
CRITIQUE	skill_lifecycle.py	Validation n'execute pas le code genere
CRITIQUE	free_router.py	G4F_MODELS reference mais non importe (NameError)
CRITIQUE	registry.py	spawn() n'invoque jamais la fabrique
CRITIQUE	openai_layer.py	Handoff natif non configure (pass)
CRITIQUE	react.py	Parametre tools accepte mais ignore
CRITIQUE	lats.py	rollout_node potentiellement non defini
CRITIQUE	gateway.py	Import ReactReasoner inexistant
CRITIQUE	screen_understanding.py	Troncature base64 b64[:10000] corrompt images
CRITIQUE	vrn_renderer.py	Serveur WebSocket jamais demarre
HAUT	working.py	__post_init__ condition jamais declenchee
HAUT	identity.py	Impossible de definir langue='en' sur merge
HAUT	chroma_service.py	update() remplace metadonnees au lieu de fusionner
HAUT	knowledge_graph.py	_rel_type_index non nettoye/mis a jour
HAUT	gateway.py	Comparaison token avec != au lieu de hmac.compare_digest

Tableau 3 : Synthese des bugs critiques et hauts

13. Recommandations Prioritaires

Sur la base de cette analyse approfondie, voici les recommandations classees par ordre de priorite pour ameliorer la qualite et la fiabilite du projet NexusAgent.

Priorite 1 - Bugs critiques a corriger

- Corriger le namespace mismatch dans l'orchestrateur memoire ('semantic' vs 'knowledge')

- Corriger le recall() WORKING pour utiliser les sessions existantes au lieu de creer un WorkingMemory vide
- Reactiver ou supprimer le code LangGraph mort dans langgraph_engine.py
- Importer G4F_MODELS dans free_router.py ou remplacer les references
- Corriger l'import ReactReasoner dans gateway.py (c'est ReActLoop)
- Corriger la troncature base64 dans screen_understanding.py
- Faire fonctionner spawn() dans le registre en appelant reellement la fabrique

Priorite 2 - Securite

- Utiliser hmac.compare_digest() pour la comparaison de tokens dans la passerelle API
- Ajouter une allowlist pour install_package ou le desactiver
- Restreindre les permissions du fichier pepper du vault
- Echapper les arguments dans search_files au lieu de les interpoler directement dans grep
- Remplacer tempfile.mktemp() par tempfile.mkstemp() partout
- Ajouter des limites de ressources Docker dans docker-compose.yml

Priorite 3 - Qualite du code

- Harmoniser pyproject.toml et requirements.txt pour des environnements coherents
- Corriger les URLs YOUR_USERNAME dans README.md
- Mettre a jour le CHANGELOG.md pour reflechir l'etat actuel du projet
- Eliminer la duplication entre ComputerUse et GUIController via heritage ou delegation
- Implementer le streaming LLM reel au lieu du stub no-op
- Supprimer le code mort (_call_glm_direct, _call_ollama_direct, _safe_json_parse)
- Creer des singletons pour LLMRouter et NexusMemoryService au lieu de les instancier par appel

Priorite 4 - Ameliorations fonctionnelles

- Implementer l'execution d'outils dans la boucle ReAct
- Ajouter un vrai cross-encoder pour le reranking dans le pipeline RAG
- Integrer le KnowledgeGraph dans le RAG pipeline
- Ajouter la detection de cycles dans la boucle de handoff
- Implementer les stubs (reason_tot, reason_lats, run_supervisor, run_swarm, web_screenshot)
- Ajouter une gestion de fenetre de contexte dans BaseAgent (truncation/summarization)
- Demarrer le serveur WebSocket dans le VRMRenderer

Priorite 5 - Architecture

- Decomposer gateway.py (~1000+ lignes) en routeurs FastAPI separes
- Remplacer le dictionnaire hardcoded de 32 outils MCP par un systeme a auto-decouverte

- Ajouter un nettoyage automatique pour les collections en memoire croissantes (_spans, _instances, _windows, etc.)
- Rendre l'URL du backend configurable dans le frontend (variable d'environnement)
- Ajouter des tests d'integration pour les chemins critiques (LLM routing, memory store/recall, sandbox execution)